

To use OrderService Beans using annotation, we can use @component annotation:

```
import org.springframework.stereotype.Component;

@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

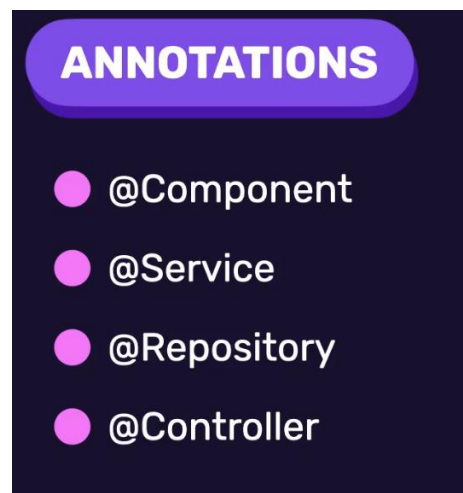
    public void placeOrder() {
        paymentService.processPayment(100_000);
    }

}
```

Service for classes that encapsulate the logic work

Repository for classes/Interfaces that handle database operations.

Service is alias for *Component*



Note 1: we set `@Component` & `@Service` for setting the Dependencies that will use

Note 2: if we have multiple constructors, then we must use `@Autowired` with classes that use `@Service` or `@Component`, because Spring prefers to use the default constructor.

Note 3: when we have multiple constructor we must set `@Autowired`, so Spring Can Handle DI Correct (Not only the default constructor) (If we don't set it we will get `BeanCreationException`).

Note: If we have two component with `@Component` OR `@Service`, then we will get exception when using the Bean Class that depends on Them.

```
@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

}
```

```
@Service
public class StripePaymentService implements PaymentService {

    @Override
    public void processPayment(double amount) {
        System.out.println("Stripe");
        System.out.println("amount: " + amount + " processed");
    }

}
```

```

@Service
public class PaypalPaymentService implements PaymentService{

    @Override
    public void processPayment(double amount) {
        System.out.println("Paypal");

        System.out.println("amount: " + amount + " processed");
    }
}
-----

@SpringBootApplication
public class StoreApplication {

    public static void main(String[] args) {
        ApplicationContext context =
        SpringApplication.run(StoreApplication.class, args);

        OrderService orderService = context.getBean(OrderService.class);
        orderService.placeOrder();
    }
}
-----

```

To solve the above problem, we have *@Primary @Qualifier*

@Primary: Set the default bean

@Qualifier: set the implementation of what we want.

In This Way we set default value for @Value, if the param not defined in **application.properties**:

@Value("\${not.defined:Hi}")

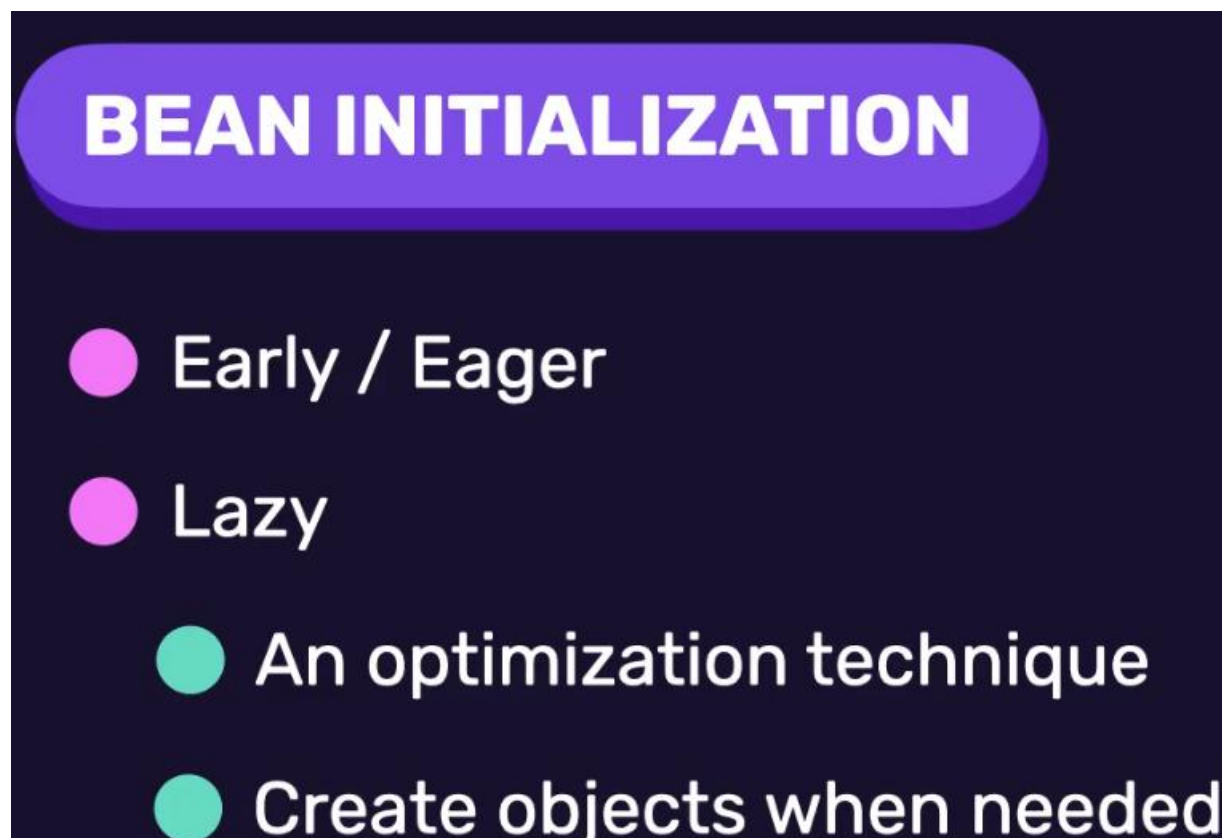
private String notDefined;

@Bean: used with method to tell springboot framework it's Bean producer

@Configuration: used with class to create configuration classes for our project, so we control the operation of DI (*springboot framework will auto load the class when run*)

For our classes that we want to create it, we must annotate it with **@Component**

Note: when use **@Configuration**, springboot will always initialize the classes, so to avoid unnecessary of creating objects (this called early or eager initialization), we can *use lazy loading*



The image is a presentation slide with a dark blue background. At the top, there is a purple rounded rectangle containing the title 'BEAN INITIALIZATION' in white, bold, uppercase letters. Below the title, there is a list of four bullet points. The first two are pink circles followed by the text 'Early / Eager' and 'Lazy'. The next two are teal circles followed by the text 'An optimization technique' and 'Create objects when needed'.

- Early / Eager
- Lazy
- An optimization technique
- Create objects when needed

After comment all code that uses the *@Component-classes*, this is the output when using eager loading:

```
2025-08-22T13:15:51.470+03:00 INFO 16948 --- [store] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat ini
2025-08-22T13:15:51.478+03:00 INFO 16948 --- [store] [main] o.apache.catalina.core.StandardService : Starting s
2025-08-22T13:15:51.478+03:00 INFO 16948 --- [store] [main] o.apache.catalina.core.StandardEngine : Starting s
2025-08-22T13:15:51.506+03:00 INFO 16948 --- [store] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializi
2025-08-22T13:15:51.506+03:00 INFO 16948 --- [store] [main] w.s.c.ServletWebServerApplicationContext : Root WebAp
HeavyResources created
NotificationManager created
OrderService created
2025-08-22T13:15:51.570+03:00 INFO 16948 --- [store] [main] o.s.b.a.w.s.WelcomePageHandlerMapping : Adding wel
2025-08-22T13:15:51.713+03:00 INFO 16948 --- [store] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat sta
2025-08-22T13:15:51.726+03:00 INFO 16948 --- [store] [main] com.jafarloka.store.StoreApplication : Started St
```

The *default scope* for services is **singleton**, so only one instance of the services will be created, and *the constructor* will be called only once.

Prototype scope: will create service for each request, it is useful for services that hold state of info about user activity and that are used temporarily.

Note 1: in this way if we call the service twice using `context.getBean`, then the constructor will apply twice.

Note 2: if we want to use it, we annotate the class, method (ex: in AppConfig-Class) with

@Scope("prototype")

The other two scopes of bean: **request** (like asp.net core), **session** (like asp.net core)

Bean Lifecycle Hooks: useful when we have many init works that depend on the values of Bean that must be initialized.

We must use *@Autowired* carefully, it may make framework call the hooks (*PostConstruct Twice*).

This is used inside the orderService class

```
@PostConstruct
public void init() {
    System.out.println("OrderService PostConstruct");
}

@PreDestroy
public void cleanup() {
    System.out.println("OrderService PreDestroyed");
}
```

To test them:

```
ConfigurableApplicationContext context =
SpringApplication.run(StoreApplication.class, args);

OrderService orderService = context.getBean(OrderService.class);
context.close();
```

A dark blue slide with a purple rounded rectangle at the top left containing the text 'JPA'. Below it, there is a list of four bullet points, each preceded by a pink circle. The bullet points are: 'Jakarta Persistence API', 'A specification for mapping Java objects to database tables', 'We don't write SQL queries', and 'We work directly with objects'.

JPA

- Jakarta Persistence API
- A specification for mapping Java objects to database tables
- We don't write SQL queries
- We work directly with objects



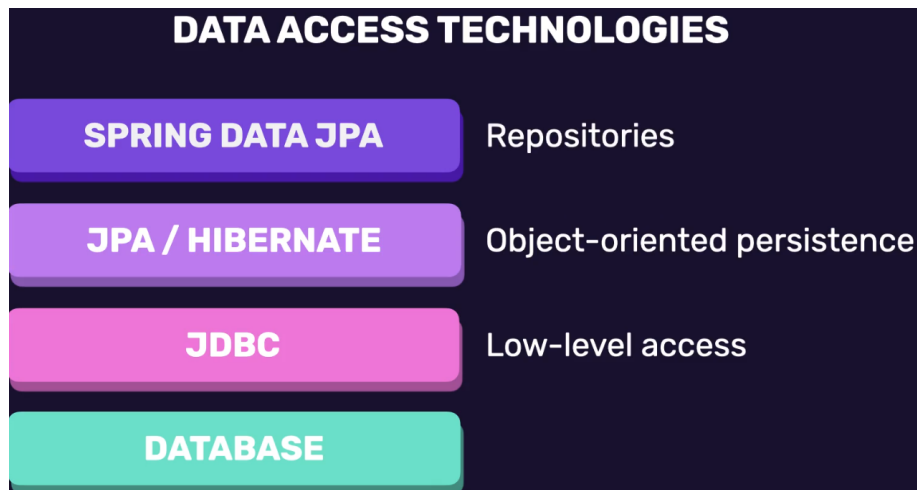
HIBERNATE

- An Object/Relational Mapper (ORM)
- Similar to Entity Framework (in .NET)
- We don't write SQL queries
- Provides caching
- Hibernate Query Language
- Automatic schema generation

SPRING DATA JPA

- Provides repository interfaces

```
public interface UserRepository extends CrudRepository<User, Long> {  
}
```



```
package com.jafarloka.store;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Lazy;

@Configuration
public class AppConfig {

    @Value("${payment-gateway:paypal}")
    private String paymentGateway;

    @Bean
    public PaymentService stripe() {
        return new StripePaymentService();
    }

    @Bean
    public PaymentService paypal() {
        return new PaypalPaymentService();
    }

    @Bean
    @Lazy
    public OrderService orderService() {
        if(paymentGateway.equals("paypal")){
            return new OrderService(paypal());
        }
        return new OrderService(stripe());
    }

    @Bean
    public NotificationService email() {
        return new EmailNotificationService();
    }
}
```



```

@Bean
public UserRepository inmMemory() {
    return new InMemoryUserRepository();
}

@Bean
public UserService userService() {
    return new UserService(inmMemory(), email());
}
}

*****

```

The pom.xml file for flyway && JPA && mariadb is:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.3</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.jafarloka</groupId>
    <artifactId>store</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>store</name>
    <description>store application</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>24</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>

        <dependency>

```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
        <version>3.5.3</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Flyway Core -->
    <dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-core</artifactId>
    </dependency>

    <dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-mysql</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.data</groupId>
        <artifactId>spring-data-jpa</artifactId>
        <version>3.5.3</version>
    </dependency>

    <dependency>
        <groupId>org.mariadb.jdbc</groupId>
        <artifactId>mariadb-java-client</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>
*****

```